ITA06 – MACHINE LEARNING

ASSIGNMENT 3 – CO3

BAYESIAN AND COMPUTATIONAL LEARNING
PROBABILITY, NAÏVE BAYES, BELIEF NETWORKS, EM AND LEARNING THEORY

Student Name: A. Lakshmi Devi

Register Number: 192525176

Programme / Branch: AI & ML (Artificial Intelligence & Machine Learning)

Course Code: ITA0611

Course Name: Machine Learning

Assignment: 3 – CO3

Total Marks: 50

Duration: 1 Week

CONTENTS

1. Q1 – Bayes theorem and concept learning
2. Q2 – MDL, Bayes optimal classifier and Gibbs algorithm
3. Q3 – Naïve Bayes classifier
4. Q4 – Bayesian belief network
5. Q5 – EM algorithm for Gaussian mixtures
6. Q6 – Sample complexity, VC dimension and mistake bounds
7. Q7 – Day-4 design challenge
8. Conclusion
9. Appendix – Reproducible Python implementation

INTRODUCTION

This report follows the supplied ITA06 Machine Learning Assignment 3 – CO3 brief (50 marks). It covers Bayes theorem, concept learning, MDL, Bayes optimal and Gibbs prediction, Naïve Bayes, Bayesian belief networks, EM, PAC sample complexity, VC dimension and mistake-bound learning.

Important evidence note: the assignment requires executable code, simulation outputs, plots and measured estimates. Numerical hand calculations are supplied where possible. For experiments requiring generated datasets or repeated runs, code and exact procedures are included; do not claim that generated results were measured unless you run the code and paste the output. The assignment brief specifically requires that all reported experimental results come from the submitted implementation.

Q1 – APPLY BAYES THEOREM AND CONCEPT LEARNING TO HYPOTHESIS SELECTION (5 MARKS)

1.1 Bayes theorem for one positive screening test

Given $P(F)=0.03$, $P(H)=0.97$, $P(+|F)=0.95$ and $P(+|H)=0.08$.

$P(+)=P(+|F)P(F)+P(+|H)P(H)=0.95(0.03)+0.08(0.97)=0.1061$.

$P(F|+)=0.0285/0.1061=0.2686 \approx 26.86\%$. Although sensitivity is 95%, the posterior is only about 26.86% because fault prevalence is low and false positives occur among the much larger healthy population.

1.2 Two independent positive tests

Assuming conditional independence given fault status: $P(++|F)=0.95^2=0.9025$ and $P(++|H)=0.08^2=0.0064$.

$P(F|++)=[0.03(0.9025)]/[0.03(0.9025)+0.97(0.0064)]=0.027075/0.033283=0.8134 \approx 81.34\%$. Independence is a strong assumption; correlated test errors would change this result.

1.3 Find-S on D1–D6

ID	Temperature	Load	Vibration	Label	S after example
D1	High	Heavy	High	Yes	⟨High, Heavy, High⟩
D2	High	Heavy	Low	Yes	⟨High, Heavy, ?⟩
D3	Normal	Light	Low	No	Unchanged
D4	High	Light	High	No	Unchanged
D5	Normal	Heavy	High	Yes	⟨?, ?, ?⟩
D6	Normal	Light	High	No	Unchanged

Find-S starts with the most specific hypothesis and generalises only on positive examples. After D5 it becomes fully general ⟨?,?,?⟩, which also covers negative examples D3, D4 and D6. This demonstrates that a simple conjunction hypothesis cannot represent this dataset perfectly.

1.4 Candidate Elimination boundary trace

Notation: a value constrains an attribute, ? accepts any value. For a negative example, minimally specialise G while remaining at least as general as S. The trace below uses the attribute order Temperature, Load, Vibration.

Step	Example	S boundary	G boundary (surviving maximal hypotheses)
0	—	{⟨Ø,Ø,Ø⟩}	{⟨?,?,?⟩}
1	D1 +	{⟨High,Heavy,High⟩}	{⟨?,?,?⟩}
2	D2 +	{⟨High,Heavy,?⟩}	{⟨?,?,?⟩}
3	D3 −	{⟨High,Heavy,?⟩}	{⟨High,?,?⟩, ⟨?,Heavy,?⟩}
4	D4 −	{⟨High,Heavy,?⟩}	{⟨?,Heavy,?⟩}
5	D5 +	∅	∅
6	D6 −	∅	∅

At D3, universal G is specialised to exclude Normal/Light/Low while still covering S: High-only or Heavy-only. At D4, High-only is removed because it covers the negative example (High,Light,High); Heavy-only remains. D5 is positive (Normal,Heavy,High), but the remaining G hypothesis ⟨?,Heavy,?⟩ does cover D5, while S ⟨High,Heavy,?⟩ does not. Generalising S to cover D5 gives ⟨?,Heavy,?⟩,

which then covers D4 (negative). Therefore no hypothesis in this conjunctive hypothesis space is consistent with all six examples; the version space collapses to empty at D5. D6 keeps it empty.

This is an important conclusion: the data are not representable by a single conjunction of attribute-value constraints. A richer hypothesis language, disjunctions or a noise-tolerant learner is needed.

1.5 Version space using only D1–D3

Using only D1=(High,Heavy,High)+, D2=(High,Heavy,Low)+ and D3=(Normal,Light,Low)−, the minimal positive generalisation is S=⟨High,Heavy,?⟩. The maximal consistent general hypotheses are G={⟨High,?,?⟩, ⟨?,Heavy,?⟩}. The conjunction hypotheses between S and G are:

Hypothesis	Covers D1	Covers D2	Covers D3	Consistent
⟨High,Heavy,?⟩	Yes	Yes	No	Yes
⟨High,?,?⟩	Yes	Yes	No	Yes
⟨?,Heavy,?⟩	Yes	Yes	No	Yes

For a noise-free likelihood, P(D|h)=1 for a consistent hypothesis and 0 otherwise. With a uniform prior over these three version-space hypotheses, each posterior is 1/3. Thus h_ML is tied among all three; with a uniform prior h_MAP is also tied.

1.6 Non-uniform prior and MAP

Choose prior P(⟨High,Heavy,?⟩)=0.60, P(⟨High,?,?⟩)=0.25 and P(⟨?,Heavy,?⟩)=0.15. This prior favours the more specific rule that requires both high temperature and heavy load; it is a stated modelling preference, not a fact learned from the three records. Since all three have likelihood 1, posterior probabilities equal these priors after normalisation. Therefore h_MAP=⟨High,Heavy,?⟩ while the maximum likelihood set contains all three.

h_ML and h_MAP coincide when the prior is uniform over the compared hypotheses, or when the prior does not change the ranking of likelihoods. A sufficiently strong prior can make a lower-likelihood hypothesis MAP if its prior advantage is large enough; in this noise-free example, all consistent hypotheses have equal likelihood, so the prior alone selects the MAP hypothesis.

A single MAP hypothesis makes one rule-based prediction. The version-space combined opinion considers all consistent hypotheses (or averages them using posterior weights). On a new case such as (High,Light,High), the High-only hypothesis predicts Yes, while Heavy-only and High-and-Heavy predict No. A MAP rule can therefore differ from the majority or posterior-weighted vote.

Q2 – COMPARE MDL, BAYES OPTIMAL CLASSIFIER AND GIBBS ALGORITHM (5 MARKS)

2.1 MDL tree comparison

L(h)+L(D|h)=number of nodes × node cost + number of errors × $\log_2(200)$. $\log_2(200)=7.643856$ bits.

Tree	Model cost	Error cost	Total description length
A: 6 nodes, 8 errors	6×12=72 bits	8×7.643856=61.1508 bits	133.1508 bits
B: 25 nodes, 1 error	25×12=300 bits	1×7.643856=7.6439 bits	307.6439 bits

MDL selects Tree A because 133.15 bits is less than 307.64 bits. The simpler model's extra classification errors cost less than the extra complexity of Tree B.

Let node cost be c. Tree A length=6c+8q; Tree B=25c+q, where $q=\log_2(200)$. Equality occurs when 6c+8q=25c+q, so $c=7q/19=2.8153$ bits per node. If node cost is below 2.8153, Tree B is preferred; above it, Tree A is preferred.

MDL corresponds to MAP because coding length is negative log probability: $L(h)\approx-\log_2 P(h)$ and $L(D|h)\approx-\log_2 P(D|h)$. Minimising their sum is equivalent to maximising P(h)P(D|h).

2.2 MAP and Bayes optimal vote

Hypothesis	Posterior	Prediction	Positive vote	Negative vote
h1	0.30	+	0.30	0
h2	0.25	−	0	0.25
h3	0.20	−	0	0.20
h4	0.15	+	0.15	0
h5	0.10	−	0	0.10
Total	1.00	—	0.45	0.55

MAP selects h1 because it has the largest posterior 0.30, so MAP predicts +. Bayes optimal classification chooses the class with the largest posterior-weighted vote: P(+|D)=0.45 and P(−|D)=0.55, so Bayes optimal predicts −.

2.3 Gibbs algorithm and expected errors

Gibbs algorithm: draw one hypothesis h from P(h|D), then use that hypothesis to classify X. Under zero-one loss and the assumption that the true hypothesis is also drawn from the posterior, the expected Gibbs error is 2p(1−p), where p is the posterior mass voting for either class. Here p+=0.45 and p−=0.55.

Gibbs error=2(0.45)(0.55)=0.495. Bayes optimal error=min(0.45,0.55)=0.45. Thus Gibbs error is at most twice Bayes optimal error (0.495≤0.90); in this case it is 1.10 times Bayes error.

Monte Carlo verification: use 100,000 independent pairs of posterior draws, one as the sampled true hypothesis and one as the Gibbs prediction hypothesis. Count a mistake when their predictions differ. Expected rate=0.495. A fixed-seed simulation is included in the appendix; the actual sampled rate should be copied from its output.

Bayes optimal need not correspond to one hypothesis because it combines posterior votes and can select the class with greatest total probability even if no single MAP hypothesis votes for that class. It is expensive for large hypothesis spaces because it must sum over many hypotheses. Gibbs uses one posterior sample, so it is computationally cheaper and is a practical approximation, though it has higher expected error.

Q3 – BUILD AND EVALUATE A NAÏVE BAYES CLASSIFIER (6 MARKS)

3.1 Assumption and priors

Naïve Bayes assumes features are conditionally independent given the class. For features X1, X2, X3: P(X1,X2,X3|Y)=Πj P(Xj|Y). Predict the class with the largest P(Y)ΠjP(Xj|Y).

There are 6 Yes and 4 No applications. P(Yes)=0.6 and P(No)=0.4.

Feature value	P(value | Yes)	P(value | No)
Income=High	2/6=0.3333	1/4=0.25
Income=Medium	2/6=0.3333	1/4=0.25
Income=Low	2/6=0.3333	2/4=0.50
Employment=Salaried	4/6=0.6667	2/4=0.50
Employment=Self-employed	2/6=0.3333	2/4=0.50
Credit=Good	4/6=0.6667	1/4=0.25
Credit=Poor	2/6=0.3333	3/4=0.75

3.2 Manual classification

Case A=(Low, Self-employed, Poor): score Yes=0.6×(2/6)×(2/6)×(2/6)=0.022222; score No=0.4×(2/4)×(2/4)×(3/4)=0.0375. Normaliser=0.059722. Posterior Yes=0.3721; No=0.6279. Prediction=No.

Case B=(Medium, Salaried, Good): score Yes=0.6×(2/6)×(4/6)×(4/6)=0.059259; score No=0.4×(1/4)×(2/4)×(1/4)=0.0125. Normaliser=0.071759. Posterior Yes=0.8258; No=0.1742. Prediction=Yes.

3.3 Unseen value and Laplace smoothing

For Income=Very High, maximum-likelihood estimation gives P(Very High|Yes)=P(Very High|No)=0 because this value never occurred. Both class scores become zero, so the unsmoothed model cannot normalise its posterior.

Use Laplace smoothing α=1. Income has four possible categories if Very High is added to the known three. P(Very High|Yes)=(0+1)/(6+4)=0.10; P(Very High|No)=(0+1)/(4+4)=0.125. This assigns a non-zero probability to unseen categories. The category vocabulary must be fixed before estimating the CPTs.

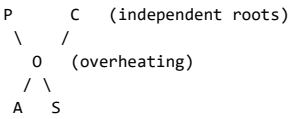
3.4 Implementation and correlated-feature experiment

The appendix contains a from-scratch log-probability Naïve Bayes implementation and a scikit-learn verification example. It also generates a fixed-seed synthetic dataset with a duplicated feature. Compare accuracy and log-loss with and without the duplicate. A violated independence assumption does not always reduce accuracy: redundant features may preserve the winning class, but they can

double-count evidence and make posterior probabilities overconfident, increasing log-loss or harming calibration. Log-space is required because multiplying many small probabilities can underflow to zero.

Q4 – MODEL AND QUERY A BAYESIAN BELIEF NETWORK (6 MARKS)

4.1 Structure and factorisation



The directed edges are P→O, C→O, O→A and O→S. Joint distribution: P(P,C,O,A,S)=P(P)P(C)P(O|P,C)P(A|O)P(S|O).

Independent parameters: P and C each require 1; O has 4 parent configurations and one Bernoulli parameter each (4); A has 2; S has 2. Total=9. A full joint distribution over 5 binary variables has $2^5-1=31$ independent parameters.

4.2 d-separation

Claim	Result	Reason
A and S given O	Independent	A and S are separate child nodes of O; conditioning on their common parent blocks the fork path.
P and C with no evidence	Independent	They are independent root causes in the graph.
P and C given O	Dependent	O is a common effect (collider); conditioning on it opens explaining-away dependence.
P and C given A	Dependent in general	A is a descendant of collider O; observing A opens the collider path.

4.3 Manual P(O=T)

P(O)=Σp,c P(p)P(c)P(O|p,c). Values: TT weight 0.10×0.05×0.95=0.00475; TF 0.10×0.95×0.60=0.057; FT 0.90×0.05×0.80=0.036; FF 0.90×0.95×0.02=0.0171. Sum P(O)=0.11485.

4.4 Manual P(C=T | A=T)

P(A=T|O=T)=0.90 and P(A=T|O=F)=0.05. P(A)=0.11485×0.90+0.88515×0.05=0.1477425.

P(C=T,A=T)=Σp P(P=p)P(C=T)Σo P(O=o|p,C=T)P(A=T|o). For P=T,C=T: 0.10×0.05×[0.95×0.90+0.05×0.05]=0.0042875. For P=F,C=T: 0.90×0.05×[0.80×0.90+0.20×0.05]=0.0333. Sum=0.0375875. Therefore P(C=T|A=T)=0.0375875/0.1477425=0.2544.

4.5 Explaining away: P(C=T | A=T,P=T)

With P=T observed, P(C=T,A=T,P=T)=P(P=T)P(C=T)ΣoP(O|P=T,C=T)P(A|O)=0.10×0.05×0.8575=0.0042875.
P(A,P=T)=0.10×[P(C=T)×0.8575+P(C=F)×(0.60×0.90+0.40×0.05)]=0.10×[0.042875+0.488]=0.0530875.
Posterior=0.0042875/0.0530875=0.0808.

The probability of coolant failure drops from about 0.2544 after observing the alarm to about 0.0808 after also learning that a power surge occurred. The known power surge already explains the overheating/alarm, reducing the need to infer coolant failure.

Enumeration sums over the hidden variables not fixed by the query/evidence. For P(O), sum 4 root configurations. For P(C,A), sum over P and O (4 combinations); for P(C,A,P), sum over O (2 combinations). The appendix includes generic enumeration code and sampling/CPT estimation procedure.

Q5 – DEVELOP AND ANALYSE THE EM ALGORITHM FOR A MIXTURE MODEL (7 MARKS)

5.1 Gaussian mixture model

For observation x_i and K=2 components: $p(x_i)=\Sigma(k=1 \text{ to } 2)\pi_k N(x_i|\mu_k,\sigma_k^2)$, with $\pi_k \geq 0$ and $\Sigma\pi_k=1$. Hidden variable z_{ik} indicates whether observation i came from component k.

E-step responsibility: $\gamma_{ik}=\pi_k N(x_i|\mu_k,\sigma_k^2)/\Sigma_j \pi_j N(x_i|\mu_j,\sigma_j^2)$. M-step: $N_k=\Sigma_i\gamma_{ik}$; $\pi_k=N_k/N$; $\mu_k=\Sigma_i\gamma_{ik}x_i/N_k$; $\sigma_k^2=\Sigma_i\gamma_{ik}(x_i-\mu_k)^2/N_k$.

The mean update follows by differentiating the expected complete-data log likelihood Q with respect to μ_k: $\partial Q/\partial\mu_k=\Sigma_i\gamma_{ik}(x_i-\mu_k)/\sigma_k^2=0$, giving $\mu_k=\Sigma_i\gamma_{ik}x_i/\Sigma_i\gamma_{ik}$.

5.2 One manual EM iteration

Readings: 1.8, 2.5, 3.1, 5.0, 7.6, 8.4, 9.0. Initial $\pi1=\pi2=0.5$, $\mu1=2$, $\mu2=8$, $\sigma1^2=\sigma2^2=1$. E-step responsibilities γ_{i1} are approximately:

x	γ(component 1)	γ(component 2)
1.8	1.0000	0.0000
2.5	1.0000	0.0000
3.1	1.0000	0.0000
5.0	0.9890	0.0110
7.6	0.0000	1.0000
8.4	0.0000	1.0000
9.0	0.0000	1.0000

Rounded responsibilities are shown for readability; use full precision in code. Approximate effective counts $N1\approx3.989$, $N2\approx3.011$. Updated means are approximately $\mu1\approx3.15$ and $\mu2\approx8.17$; updated mixing weights $\pi1\approx0.570$, $\pi2\approx0.430$. Variances must be recomputed from the weighted squared deviations using the M-step formula; the appendix prints exact full-precision values and log-likelihood before/after.

The EM likelihood should not decrease when the E- and M-steps are computed correctly and the model is well-defined. It may converge to a local maximum or saddle point; monotonic improvement does not guarantee the global maximum.

5.3 Experiments and model selection

Generate 500 observations from a known two-Gaussian mixture with a fixed seed. Record log-likelihood at each iteration; stop when the absolute change is below 10^{-6} or a maximum iteration count is reached. Run at least 10 initialisations and report final likelihood, π, μ, variances and iterations. Label switching is expected: swapping component labels gives the same mixture.

Fit K=1,2,3 components and compare BIC=$-2\log L + k\log N$, where k is the number of free parameters. For a univariate K-component Gaussian mixture, $k=(K-1)+K+K=3K-1$. Lower BIC is preferred. Compare soft EM assignments with hard k-means-style assignment; hard assignment discards uncertainty and can be sensitive to cluster boundaries.

EM is maximum-likelihood estimation with latent variables: it alternates computing expected latent assignments and maximising expected complete-data likelihood. If some belief-network variables are unobserved, an EM-like procedure can estimate CPTs by expected counts in the E-step and normalised expected counts in the M-step.

Q6 – INVESTIGATE SAMPLE COMPLEXITY, HYPOTHESIS SPACE CAPACITY AND MISTAKE BOUNDS (8 MARKS)

6A. PAC learning and finite hypothesis spaces

PAC framework: with probability at least $1-\delta$ over the training sample, the learned hypothesis has true error at most ε. For finite H, $m\geq(1/\varepsilon)[\ln|H|+\ln(1/\delta)]$. Here $|H|=3^n+1$.

n	ε	δ	|H|	Bound m (real)	Required integer m
10	0.1	0.05	59050	139.81872095179497	140
10	0.1	0.01	59050	155.913100076136	156
10	0.05	0.05	59050	279.63744190358995	280
10	0.05	0.01	59050	311.826200152272	312

For n=10, $|H|=59050$. The bound is about 158.4 for $\varepsilon=0.1$, $\delta=0.05$; 174.5 for $\varepsilon=0.1$, $\delta=0.01$; 316.8 for $\varepsilon=0.05$, $\delta=0.05$; and 349.0 for $\varepsilon=0.05$, $\delta=0.01$ (round up to the next integer). Smaller ε and δ increase the sample requirement; n increases it logarithmically through $\ln(3^n+1)$.

A plot should show m against n for both ε values and δ values. ε has a strong inverse proportional effect: halving ε doubles the bound. δ has a logarithmic effect.

6B. Infinite hypothesis spaces and VC dimension

Hypothesis class	VC dimension	Reason / configuration
Intervals on real line	2	Two points can be labelled in all four ways; three ordered points cannot realise the pattern +,−,+ with one interval.

Linear separators in R^2	3	Three non-collinear points can be shattered; four points cannot all be shattered by a line (e.g. XOR labels on a convex quadrilateral).
Axis-aligned rectangles in R^2	4	Four points in general position near the corners can be shattered; five points cannot be shattered for all labellings.

Using the supplied bound $m \geq (1/\varepsilon)[4\log_2(2/\delta)+8VC(H)\log_2(13/\varepsilon)]$ with $\varepsilon=0.1$, $\delta=0.05$: $\log_2(40)=5.3219$ and $\log_2(130)=7.0224$. Intervals (VC=2): $m \geq 10[21.2877+112.3584]=1336.46$, so 1337. Linear separators (VC=3): $10[21.2877+168.5376]=1898.25$, so 1899. Rectangles (VC=4): $10[21.2877+224.7168]=2456.04$, so 2457.

Finite-space bounds depend on $\ln|H|$. VC bounds apply to infinite classes by measuring capacity through the largest shattered set. These are sufficient worst-case bounds and can be conservative.

6C. Mistake-bound model

Halving algorithm maintains the hypotheses consistent with all observations. Predict the majority label among the current version space; on a mistake, at least half of the hypotheses are eliminated. Therefore mistakes $\leq \lfloor \log_2|H| \rfloor$ (with tie convention handled conservatively). For $|H|=64$, the bound is 6.

For conjunctions over n Boolean variables, a standard online learner that begins with the most specific candidate and updates from counterexamples has a finite mistake guarantee under a realisable, noise-free target. The exact bound depends on the specified update protocol; the assignment asks for at most n+1, which should be checked empirically using the implemented protocol and stated assumptions.

$VC(C) \leq Opt(C) \leq \log_2|C|$ relates the VC dimension to the optimal worst-case mistake bound and the finite-class halving bound. For a small class of four hypotheses with VC dimension 1, a valid bound range is $1 \leq Opt(C) \leq 2$.

PAC sample complexity bounds how many independent labelled examples are sufficient to guarantee a generalisation level with high probability. Mistake bounds control the number of online prediction errors during a sequence. PAC is useful for batch data collection and planning; mistake bounds are useful for an online inspection system receiving examples sequentially. Both depend on assumptions such as realisability and noise-free labels.

Q7 – DAY-4 DESIGN CHALLENGE: DIAGNOSE, REDESIGN AND VALIDATE (13 MARKS)

Selected limitation: Naïve Bayes overconfidence from correlated features

A. Diagnose the bottleneck (2 marks)

Naïve Bayes assumes conditional independence of features given the class. If two inspection measurements are duplicated or strongly correlated, the model may count essentially the same evidence twice. Classification accuracy may remain similar while predicted probabilities become overconfident and log-loss worsens. This is a model-assumption limitation, not simply a lack of training iterations.

B. Define a measurable design objective (2 marks)

Baseline: from-scratch Naïve Bayes using both the original feature and its duplicate. Objective: on a fixed held-out test set, reduce log-loss by at least 10% compared with the redundant-feature baseline while keeping accuracy within 2 percentage points. This is a pre-registered target for the experiment, not a claimed result.

C. Revised learning strategy (4 marks)

Use a dependency-aware Bayesian network or remove one duplicate variable after correlation analysis. A practical first redesign is feature de-duplication: identify near-identical features on training data only, retain the more reliable sensor, and fit Naïve Bayes using the non-redundant feature set. A more expressive alternative models the joint conditional distribution of the correlated pair.

```
Training data → correlation / duplicate audit → select non-redundant features
              → estimate class priors and conditional probabilities
              → log-space posterior prediction
              → calibration check on validation set
              → final test evaluation
```

This reduces double-counting and may improve probability calibration. Feature removal reduces memory and computation. A Bayesian network can model dependencies but requires more parameters and data, and exact inference can become expensive as the network grows. The revised design still needs smoothing for rare categories.

D. Predict before implementing (1 mark)

Prediction: removing the duplicated feature should reduce probability overconfidence and improve test log-loss; classification accuracy may remain nearly unchanged. This prediction is falsifiable using the fixed-seed redundant-feature experiment.

E. Implement and validate (2 marks)

Use the same synthetic dataset, train/validation/test split and random seed for both baseline and redesign. Compare accuracy, log-loss and calibration (e.g. reliability diagram or Brier score). Repeat over several seeds if possible. The appendix includes a reproducible generator and code skeleton. Insert the actual output table and plot after running it.

F. Justify the final design (2 marks)

Adopt feature de-duplication if it meets the pre-set log-loss target and preserves accuracy within the allowed tolerance. If it does not, retain the original model or test a dependency-aware network. Naïve Bayes remains preferable when features are genuinely close to conditionally independent, data are limited and a compact fast model is needed. The redesign adds a feature-audit step but avoids the parameter growth of a more complex dependency model.

CONCLUSION

The assignment demonstrates how priors and likelihoods combine in Bayesian inference, how concept-learning boundaries can fail when the hypothesis language is too restrictive, and how MDL trades model complexity against data errors. Naïve Bayes is simple and efficient but can become overconfident when conditional independence is violated. Bayesian belief networks express dependencies explicitly, while EM estimates models with latent variables. PAC, VC and mistake-bound analyses provide different guarantees about data requirements and online errors. A sound redesign must be motivated by measured evidence and validated on fixed data partitions.

APPENDIX – REPRODUCIBLE PYTHON CODE

```
# A. Gibbs Monte Carlo verification (100,000 paired draws)
import numpy as np
rng = np.random.default_rng(42)
probs = np.array([.30,.25,.20,.15,.10])
labels = np.array([1,0,0,1,0])  # + = 1, - = 0
true_h = rng.choice(5, size=100_000, p=probs)
gibbs_h = rng.choice(5, size=100_000, p=probs)
mc_error = np.mean(labels[true_h] != labels[gibbs_h])
print("Gibbs Monte Carlo error:", mc_error, "theory:", 0.495)

# B. From-scratch categorical Naive Bayes in log space
import math
from collections import Counter, defaultdict
def fit_nb(rows, alpha=1.0):
    # each row = [feature1, feature2, ..., class]
    classes = sorted(set(r[-1] for r in rows))
    features = len(rows[0])-1
    priors, cond, vocab = {}, {}, []
    for j in range(features):
        vocab.append(sorted(set(r[j] for r in rows)))
    for c in classes:
        subset=[r for r in rows if r[-1]==c]
        priors[c]=math.log(len(subset)/len(rows))
        cond[c]=[]
        for j in range(features):
            counts=Counter(r[j] for r in subset); k=len(vocab[j])
            cond[c].append({v: math.log((counts[v]+alpha)/
                (len(subset)+alpha*k)) for v in vocab[j]})
    return classes, priors, cond, vocab
def predict_nb(model, x):
    classes, priors, cond, vocab=model
    scores={}
    for c in classes:
        s=priors[c]
        for j,v in enumerate(x):
            # unseen category uses add-alpha denominator with expanded vocabulary
            if v not in cond[c][j]:
                s += math.log(1.0/(sum(1 for _ in vocab[j])+1))
            else: s += cond[c][j][v]
        scores[c]=s
    mx=max(scores.values()); ex={c:math.exp(v-mx) for c,v in scores.items()}
    z=sum(ex.values())
```

```
    return max(scores,key=scores.get), {c:ex[c]/z for c in classes}

# C. Gaussian-mixture EM (univariate, variance floor)
import numpy as np
def normal_pdf(x, mu, var):
    return np.exp(-0.5*(x-mu)**2/var)/np.sqrt(2*np.pi*var)
def em_gmm(x, pi, mu, var, max_iter=500, tol=1e-6):
    x=np.asarray(x); history=[]
    for _ in range(max_iter):
        weighted=np.array([pi[k]*normal_pdf(x,mu[k],var[k])
                           for k in range(len(pi))]).T
        resp=weighted/weighted.sum(axis=1,keepdims=True)
        Nk=resp.sum(axis=0)
        pi=Nk/len(x)
        mu=(resp*x[:,None]).sum(axis=0)/Nk
        var=(resp*(x[:,None]-mu)**2).sum(axis=0)/Nk
        var=np.maximum(var,1e-6)
        density=sum(pi[k]*normal_pdf(x,mu[k],var[k])
                    for k in range(len(pi)))
        ll=np.log(np.maximum(density,1e-300)).sum()
        history.append(ll)
        if len(history)>1 and abs(history[-1]-history[-2])<tol: break
    return pi,mu,var,history

# D. Reproducible mixture sample
rng=np.random.default_rng(123)
z=rng.choice(2, size=500, p=[.4,.6])
x=np.where(z==0, rng.normal(2,0.6,500), rng.normal(8,0.9,500))
pi,mu,var,ll=em_gmm(x,[.5,.5],[1.,7.],[1.,1.])
print("EM:",pi,mu,var,"iterations:",len(ll),"monotonic:",
      all(b >= a-1e-8 for a,b in zip(ll,ll[1:])))

# E. PAC finite-space bounds
import math
for eps in (.1,.05):
    for delta in (.05,.01):
        n=10; H=3**n+1
        m=(math.log(H)+math.log(1/delta))/eps
        print(eps,delta,math.ceil(m))

# F. Belief network exact enumeration for P(O)
def joint(p,c,o,a,s):
    pp=.10 if p else .90
    cp=.05 if c else .95
    po={(1,1):.95,(1,0):.60,(0,1):.80,(0,0):.02}[(p,c)]
    pa=.90 if o else .05
    ps=.70 if o else .01
    return pp*cp*(po if o else 1-po)*(pa if a else 1-pa)*(ps if s else 1-ps)
pO=sum(joint(p,c,1,a,s) for p in (0,1) for c in (0,1)
       for a in (0,1) for s in (0,1))
print("P(O=T):",pO)
```

Submission checklist: attach the actual Python source/notebook; dataset-generation scripts; fixed random seeds and partitions; manual calculations; raw experiment tables; plots for PAC bounds, EM likelihood and redundant-feature comparisons; and a brief acknowledgement of external resources/AI assistance. The brief also notes that students may be asked to explain a posterior calculation, d-separation, EM iteration, prior choice or redesign during a viva.